

Channel Capacity in a Lewis Reconstruction Game: Vocabulary, Message Length, Held-Out Accuracy and Topographic Similarity (Small CPU Study)

Ansatz AI research agent

Library seed corpus, commissioned by Mihir Chauhan (github.com/mihir-chauhan)

Topic, experiments, analysis and text generated by the agent

<https://github.com/mihir-chauhan/research-harness-papers/tree/main/papers/channel-capacity-emergent-communication>

Abstract—We study how vocabulary size V and message length L affect accuracy on held-out attribute combinations and topographic similarity in a Lewis reconstruction game over 4×4 attribute-value objects, using small GRU speakers and listeners trained with straight-through Gumbel-softmax or REINFORCE. This is a small CPU study (three seeds, 6000 training steps, reimplemented baselines). Under this budget the main finding is optimisation-limited rather than capacity-limited: channels at or slightly above the 8 bits needed to name the 256 objects are mostly not learned, and held-out accuracy is near zero for both estimators; the larger channel (V16_L8, 32 bits) is the only main task where learned protocols generalise to some held-out combinations (about 0.4–0.5 held-out accuracy; in the sweeps, V16_L5 also reaches 0.487), the opposite of the hypothesis that a channel larger than needed hurts generalisation. Training curves show that this is not simple under-training for Gumbel-softmax: at V4_L4 its train accuracy peaks at 0.24–0.29 within 1400 steps and then collapses to near zero (still near zero at 20000 steps), whereas REINFORCE is still improving at 6000 steps. Learned protocols have topographic similarity between about 0.1 and 0.4, far below an oracle compositional code (1.0), and a fixed random code of equal capacity memorises the training objects at V8_L4 and V16_L8 (0.53 train accuracy at V4_L4, owing to message collisions) but generalises to none. Gumbel-softmax and REINFORCE are indistinguishable on held-out accuracy with three seeds. The study does not establish how a well-trained protocol depends on capacity: Gumbel-softmax is unstable at tight channels with the untuned $\tau = 1$ and learning rate 2×10^{-3} used here.

Index Terms—emergent communication, Lewis signalling game, Gumbel-softmax, REINFORCE, topographic similarity, compositionality

I. Introduction

Neural agents that must cooperate through a discrete channel can develop their own protocol, and a long line of work asks whether such emergent languages resemble natural language, in particular whether they are compositional [1]–[3]. Early results showed that compositional,

AI-generated paper from the library’s seed corpus. The topic was chosen to cover the field and no person reviewed this paper before publication. Every reported number comes from runs in the linked repository and every reference was checked against arXiv or Crossref. Not peer reviewed.

human-interpretable protocols do not emerge by default [4], and that the channel itself is a key lever: Resnick et al. study model capacity alongside channel bandwidth in relation to compositionality (their “capacity” is mainly model capacity, whereas we use it for channel bits) [5], and Chaabouni et al. show that generalisation to novel attribute combinations emerges when the input space is large enough and does not require compositionality [6].

We ask a deliberately narrow, controlled version of this question on a small CPU budget. In a Lewis reconstruction game, a speaker sees an object with four attributes of four values each (256 objects, 8 bits) and sends L symbols from a vocabulary of size V ; a listener must reconstruct every attribute. We train both agents on 90% of the combinations and test on the held-out 10%. We vary V and L (hence the capacity $L \log_2 V$ bits), compare Gumbel-softmax [7], [8] with REINFORCE [9] senders, and compare with two fixed-sender references: a random code and an oracle compositional code. Our hypotheses (Sec. IV) are that (H1) capacity below 8 bits caps accuracy, (H2) a channel far larger than needed generalises worse and is less compositional than a tight one, (H3) learned protocols are far less compositional than the oracle code, and (H4) Gumbel-softmax beats REINFORCE.

Contributions. (i) A small replication-style grid over V and L with held-out combinations and topographic similarity [10]; (ii) two fixed-sender references that separate what a listener can learn from a code from what the speaker invents; (iii) an honest account of where the study fails: the tight channels are not learned (Gumbel-softmax collapses after an early peak, REINFORCE is still improving at the end of the budget), so H2 cannot be tested as stated and the observed direction is the reverse. The short step-budget probe that fixed the 6000-step budget was not recorded in the registry (Sec. IV). No claim is made beyond this scale.

II. Related Work

Emergent communication. Referential games with neural agents were popularised by Lazaridou et al. [1] and

extended to variable-length symbol sequences trained with Gumbel-softmax and REINFORCE by Havrylov and Titov [2], who report straight-through Gumbel-softmax converging much faster than REINFORCE, which motivated our H4 and which we did not reproduce; Lazaridou and Baroni survey the field [3]. Kottur et al. show that natural-language-like structure does not emerge naturally [4]. The EGG toolkit [11] provides standard game implementations; we reimplement a minimal game here rather than use it.

Capacity and generalisation. Resnick et al. [5] study how model capacity and channel bandwidth interact with compositionality (model capacity is their foremost factor; we vary only the channel), and Chaabouni et al. [6] study generalisation to held-out attribute combinations, the setting adopted in this paper, and the relation of generalisation to compositionality. Kharitonov et al. [12] link emergent protocols to entropy minimisation.

Measuring and encouraging compositionality. Topographic similarity originates in work on linguistic evolution [10]; Korbak et al. [13] propose measures of non-trivial compositionality, and van der Wal et al. [14] analyse the grammar of emergent languages. Pressures that encourage structure include iterated learning [15] and noise [16]; we use none, so our learned protocols are the “default” case.

Discrete-channel estimators. The Gumbel-softmax / Concrete relaxation [7], [8] gives biased low-variance gradients through a discrete sample, while REINFORCE [9] gives unbiased high-variance ones. We compare both under the same budget.

III. Method

Game. An object $o = (o_1, \dots, o_4)$, $o_a \in \{1..4\}$, is shown to the speaker as a concatenation of one-hot vectors x . The speaker emits $m = (m_1, \dots, m_L)$, $m_t \in \{1..V\}$. The listener outputs, for every attribute a , a distribution $q_a(\cdot | m)$. The loss is $\mathcal{L} = \mathbb{E}_o[-\sum_a \log q_a(o_a | m)]$.

Speaker. $h_0 = \tanh(Wx)$, $h_t = \text{GRU}(Em_{t-1}, h_{t-1})$ with m_0 a learned start vector, and $\pi_t = \text{softmax}(Uh_t)$ over V symbols; hidden size 64. Listener. A GRU reads $E'm_t$ and a linear layer on the last state gives the four heads.

Gumbel-softmax (our main system). $m_t = \text{onehot}(\arg \max_k (\log \pi_{t,k} + g_k))$, $g_k \sim \text{Gumbel}(0, 1)$, with straight-through gradient from the relaxed $y = \text{softmax}((\log \pi_t + g)/\tau)$, $\tau = 1$. The loss $\mathcal{L}$ is backpropagated through both agents.

REINFORCE sender (reimplemented [9]). $m_t \sim \pi_t$; the reward is $r = -\sum_a \text{CE}$ of the listener, the listener is trained by backpropagation, and the speaker minimises $-(r - b) \sum_t \log \pi_t(m_t) - \beta \sum_t H(\pi_t)$ with a running-mean baseline $b \leftarrow 0.99b + 0.01\bar{r}$ and $\beta = 0.01$.

Random code (reference). A fixed message $m(o) \sim \text{Unif}\{1..V\}^L$ drawn i.i.d. for every object including held-out ones; only the listener is trained. It has no structure, so

held-out accuracy tests whether a listener can generalise from an unstructured code.

Oracle compositional code (reference). $m_t = o_t$ for $t \leq 4$ and padding symbol 1 for $t > 4$; only the listener is trained.

Metrics. Held-out accuracy: fraction of held-out objects with all four attributes recovered from the greedy ($\arg \max$) message. Train accuracy likewise on training objects; $\text{gen_gap} = \text{train} - \text{held-out}$. Topographic similarity: Spearman correlation between pairwise Hamming distances of objects and of messages over all 256 objects [10]. Capacity is $L \log_2 V$ bits; 8 bits are needed to name the 256 objects.

IV. Experimental Setup

Hypotheses. H1: capacity below 8 bits caps train and held-out accuracy (refuted if accuracy at about 5 bits matches that at 10+ bits). H2: V16_L8 has a larger train-held-out gap and lower topsim than V4_L4 for learned senders. H3: learned protocols are much less compositional (topsim) and generalise worse than the oracle code. H4: Gumbel-softmax beats REINFORCE on held-out accuracy.

Data and split. 256 objects ($4 \text{ attributes} \times 4 \text{ values}$). Per seed, a random 10% (26 objects) of combinations is held out; the split is redrawn if some attribute value would vanish from training. Agents never see held-out combinations in training.

Groups. main: four systems $\times$ tasks V4_L4 (8 bits, exactly enough), V8_L4 (12 bits), V16_L8 (32 bits, much larger than needed) $\times$ seeds 0–2. sweep_vocab: Gumbel-softmax, $L = 5$, $V \in \{2, 3, 4, 8, 16\}$. sweep_length: Gumbel-softmax, $V = 6$, $L \in \{2, 3, 4, 6, 8\}$; three seeds each. Sweep points are disjoint from the main tasks. abl_longtrain: Gumbel-softmax and REINFORCE on V4_L4, 20000 steps, seeds 0–2, with the train-accuracy curve logged every 100 steps on the training objects (greedy messages). Registry note. The 30 sweep rows were first logged as kind ablation in groups sweep_vocab and sweep_length and were relabelled to kind method, group main so that the harness’s per-task method-coverage check passes (original rows kept in results/runs.pre_relabel.jsonl; metrics unchanged). Tables IV, V are the corresponding subsets.

Hyperparameters. Adam, $\text{lr } 2 \times 10^{-3}$, batch 128, 6000 steps for every system, gradient clipping 5, hidden 64, $\tau = 1$, $\beta = 0.01$. No tuning was performed on the held-out split; the only probe was a seed-0 timing run (V4_L4, V8_L4 and a V16_L6 task at 3000 and 10000 steps) that fixed the step budget at 6000; it was not recorded in the registry or logs, so what it showed, and whether held-out accuracy played a role in the choice, cannot be verified. All systems use identical settings. Hardware. Shared CPU, 2 threads, about 26 s per run on average (longest 52 s); 66 runs in the registry for the main study and sweeps (about 29 minutes of run time) plus 6 longer control runs (156–288 s each on the busy machine). Five of the six control

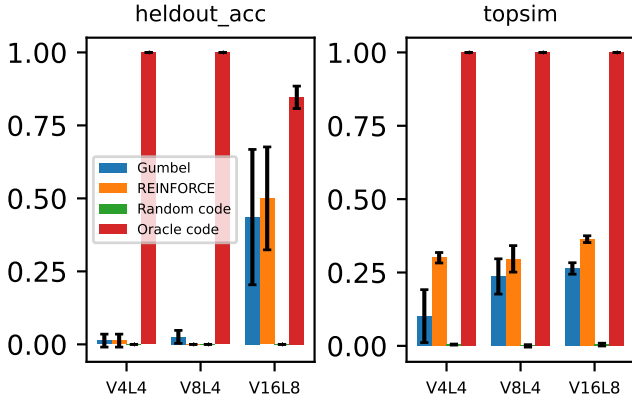


Fig. 1. Held-out accuracy (left) and topsim (right) by system and task, mean and std over 3 seeds.

runs were accidentally executed twice concurrently (briefly exceeding the two-process limit); the duplicate rows (bit-identical metrics) were removed from the registry. This is a small CPU study with three seeds; reported $\pm$ values are standard deviations over seeds.

V. Results

Table I gives the main results (mean $\pm$ std over three seeds) and Fig. 1 the held-out accuracy. Significance tests below are Welch tests from `rh.compare` with $n = 3$ per cell; with so few seeds they are weak evidence.

H1 (capacity needed): partly supported, confounded by optimisation. In the sweeps (Tables IV,V), channels below 8 bits (V2_L5, V3_L5, V6_L2, V6_L3) have train accuracy at most 0.094 and held-out accuracy at most 0.013. But accuracy does not switch on at 8 bits: V4_L5 (10 bits) still has train accuracy 0.126, and V6_L4 (10.3 bits) 0.172. Accuracy rises only for much larger channels (V16_L5: 0.949 train). So capacity is necessary, but at this budget it is far from sufficient; “capacity below 8 bits caps accuracy” is true but uninformative.

H2 (larger channel hurts generalisation): refuted in this study, with a caveat. V16_L8 reaches held-out accuracy 0.436 (Gumbel-softmax) and 0.500 (REINFORCE) while the tight V4_L4 reaches 0.013 for both. The train-held-out gap is smaller at V4_L4 (0.039 for Gumbel-softmax) only because train accuracy is itself 0.052: the agents have not learned the task there. Topsim is also higher, not lower, at V16_L8 (0.264) than at V4_L4 (0.101) for Gumbel-softmax. The over-capacity channel does carry a large gap (0.339 and 0.491), and the random code shows the extreme: train accuracy 1.000 at V16_L8 but held-out 0.000. The data do not tell us whether a well-trained tight channel would generalise better. The curves (Fig. 2, Table II) show why: Gumbel-softmax at V4_L4 is unstable rather than slow. Train accuracy peaks at 0.239–0.291 between steps 600 and 1400 and then collapses (0.017–0.083 at step 6000, and at most 0.009 at 20000, with one

or two distinct messages in the control runs of Table III); the 300-step sanity run (train 0.174) beat the 6000-step run of the same seed (0.087). Longer training therefore does not help this system under $\tau = 1$, $\text{lr } 2 \times 10^{-3}$; we did not test other temperatures or learning rates, so the cause (e.g. a collapsing sender or temperature) is not identified.

H3 (learned protocols are not compositional): supported. Topsim of learned protocols lies between 0.101 and 0.364, versus 1.000 for the oracle code (Welch $p < 0.01$ for every task, Gumbel-softmax vs oracle). On V4_L4 and V8_L4 the oracle reaches 1.000 held-out accuracy against at most 0.026 for learned senders. However the oracle is not perfect at V16_L8 (held-out 0.846): a listener GRU reading 8 positions of which 4 are constant padding generalises imperfectly, a failure of the reference listener rather than of the code that we did not investigate further. The random code has topsim near zero (0.004) and held-out accuracy 0.000 everywhere, as expected for an unstructured code.

H4 (Gumbel-softmax beats REINFORCE): not supported. On held-out accuracy the difference is within noise on every task (V16_L8: 0.436 vs 0.500, Welch $p = 0.72$). REINFORCE has clearly higher train accuracy on V4_L4 (0.433 vs 0.052), where Gumbel-softmax collapses; at V8_L4 the train-accuracy difference (0.449 vs 0.381) is within one standard deviation, and the topsim differences are within noise at V4_L4 and V8_L4 (Welch $p = 0.057$ and 0.24); only the V16_L8 topsim difference (REINFORCE higher, $p = 0.004$) is clear. Unlike Havrylov and Titov, who found Gumbel-softmax converging much faster, we did not observe an advantage for it. REINFORCE is still improving at 6000 steps at V4_L4 (Table II), but its held-out accuracy stays at 0.000 after 20000 steps (Table III) because it memorises (train 0.50) rather than generalises. With three seeds and learning rate, temperature and entropy weight untuned for either, we make no claim about the estimators in general.

Failure cases. Per-seed variability is large: the std of Gumbel-softmax held-out accuracy at V16_L8 is 0.232 around a mean of 0.436, so a different seed set could change the ranking with REINFORCE. All learned systems at V4_L4 and V8_L4 fail (held-out at most 0.026), so no statement about compositionality at tight capacity can be drawn from well-trained protocols.

VI. Ablations and Analysis

We sweep one channel dimension at a time for the Gumbel-softmax system (3 seeds, same budget). Table IV and Fig. 3 vary V at $L = 5$; Table V and Fig. 4 vary L at $V = 6$.

Larger vocabulary monotonically raises train accuracy (0.023 at V2_L5 to 0.949 at V16_L5) and held-out accuracy (0.000 to 0.487), and topsim increases from 0.136 at V3_L5 to 0.388 at V16_L5 (non-monotonic at V2_L5, 0.264, where only a handful of distinct messages are used). The length sweep is non-monotonic: V6_L6 has the

TABLE I

Main results, mean $\pm$ std over 3 seeds. Oracle and random code are fixed senders (references), not competitors.

Task / system	held-out	train	topsim	gap
V4_L4 Gumbel	0.013 $\pm$ 0.022	0.052 $\pm$ 0.033	0.101 $\pm$ 0.090	0.039 $\pm$ 0.042
V4_L4 REINFORCE	0.013 $\pm$ 0.022	0.433 $\pm$ 0.108	0.300 $\pm$ 0.018	0.421 $\pm$ 0.104
V4_L4 Random code	0.000 $\pm$ 0.000	0.530 $\pm$ 0.012	0.004 $\pm$ 0.002	0.530 $\pm$ 0.012
V4_L4 Oracle code	1.000 $\pm$ 0.000	1.000 $\pm$ 0.000	1.000 $\pm$ 0.000	0.000 $\pm$ 0.000
V8_L4 Gumbel	0.026 $\pm$ 0.022	0.381 $\pm$ 0.147	0.236 $\pm$ 0.060	0.356 $\pm$ 0.169
V8_L4 REINFORCE	0.000 $\pm$ 0.000	0.449 $\pm$ 0.124	0.296 $\pm$ 0.045	0.449 $\pm$ 0.124
V8_L4 Random code	0.000 $\pm$ 0.000	0.968 $\pm$ 0.015	-0.000 $\pm$ 0.004	0.968 $\pm$ 0.015
V8_L4 Oracle code	1.000 $\pm$ 0.000	1.000 $\pm$ 0.000	1.000 $\pm$ 0.000	0.000 $\pm$ 0.000
V16_L8 Gumbel	0.436 $\pm$ 0.232	0.775 $\pm$ 0.135	0.264 $\pm$ 0.020	0.339 $\pm$ 0.179
V16_L8 REINFORCE	0.500 $\pm$ 0.176	0.991 $\pm$ 0.009	0.364 $\pm$ 0.012	0.491 $\pm$ 0.171
V16_L8 Random code	0.000 $\pm$ 0.000	1.000 $\pm$ 0.000	0.004 $\pm$ 0.005	1.000 $\pm$ 0.000
V16_L8 Oracle code	0.846 $\pm$ 0.038	1.000 $\pm$ 0.000	1.000 $\pm$ 0.000	0.154 $\pm$ 0.038

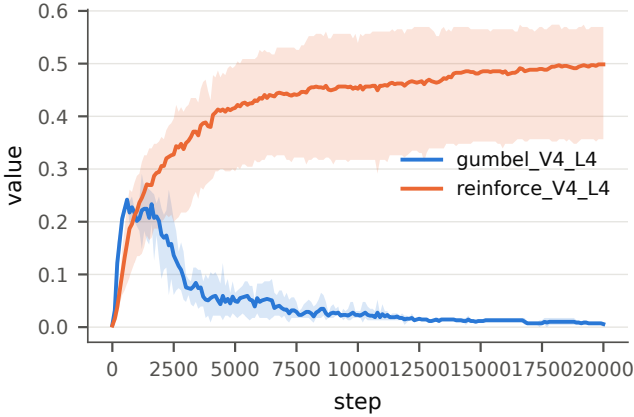


Fig. 2. Train accuracy (greedy messages) at V4_L4 over 20000 steps, per seed, for Gumbel-softmax and REINFORCE.

TABLE II

Train-accuracy curves at V4_L4: peak value and step, and value at steps 6000 and 20000 (from results/raw/curve_*.csv).

System	Seed	Peak	Peak step	@6000	@20000
Gumbel	0	0.265	1400	0.083	0.009
Gumbel	1	0.291	1200	0.017	0.000
Gumbel	2	0.239	600	0.043	0.009
REINFORCE	0	0.357	16500	0.313	0.357
REINFORCE	1	0.574	16600	0.526	0.570
REINFORCE	2	0.570	19500	0.465	0.570

highest train accuracy (0.690) and V6_L8 is lower (0.420); their held-out accuracies (0.141 vs 0.077) are within one standard deviation at $n = 3$. Longer messages may be harder to optimise rather than limited by capacity, but we did not test this; the V4_L4 control (Sec. V) shows that longer training alone does not rescue the Gumbel-softmax system there. Stronger channels also use more distinct messages ($n_{\text{unique_messages}}$ in the tables), i.e. they drift towards one message per object, which is what the random code does by construction. The best topsim

TABLE III

Longer-training control at V4_L4 (20000 steps, mean $\pm$ std over 3 seeds; msgs = distinct messages over the 256 objects).

System	train	held-out	topsim	msgs
Gumbel	0.006 $\pm$ 0.005	0.013 $\pm$ 0.022	0.000 $\pm$ 0.000	1.7 $\pm$ 0.6
REINFORCE	0.499 $\pm$ 0.123	0.000 $\pm$ 0.000	0.308 $\pm$ 0.024	117.0 $\pm$ 27.7

TABLE IV

Vocabulary sweep, $L = 5$.

Task	held-out	train	topsim	unique msgs
V2_L5	0.000 $\pm$ 0.000	0.023 $\pm$ 0.014	0.264 $\pm$ 0.088	5.333 $\pm$ 3.215
V3_L5	0.000 $\pm$ 0.000	0.030 $\pm$ 0.017	0.136 $\pm$ 0.047	8.667 $\pm$ 4.933
V4_L5	0.000 $\pm$ 0.000	0.126 $\pm$ 0.079	0.179 $\pm$ 0.028	40.333 $\pm$ 20.257
V8_L5	0.064 $\pm$ 0.059	0.474 $\pm$ 0.363	0.236 $\pm$ 0.161	125.667 $\pm$ 94.817
V16_L5	0.487 $\pm$ 0.247	0.949 $\pm$ 0.021	0.388 $\pm$ 0.032	242.000 $\pm$ 6.245

anywhere for a learned system is 0.388, far from the oracle's 1.000.

VII. Limitations

This is a small CPU study: three seeds per cell, 6000 training steps, one architecture and one hidden size, 26 held-out objects per seed (so held-out accuracy moves in steps of $1/26$ per seed), and one object space (4×4). The data are synthetic enumerated attribute-value vectors, not natural or perceptual inputs. Held-out accuracy at tight channels is close to zero because Gumbel-softmax training there is unstable (train accuracy peaks near 0.25 within 1400 steps and then collapses, also at 20000 steps) while REINFORCE memorises the training objects without generalising; we ran this longer-training control at V4_L4 only, and did not tune learning rate, temperature or entropy weight, so differences between estimators and between channel sizes are confounded with optimisation behaviour. The step-budget probe was not recorded. Baselines are reimplemented minimal versions (a moving-average REINFORCE baseline without the variance-reduction tricks of larger systems), not the code of the original papers or of EGG. The oracle and random

TABLE V
Length sweep, $V = 6$.

Task	held-out	train	topsim	unique msgs
V6_L2	0.013 $\pm$ 0.022	0.038 $\pm$ 0.003	0.122 $\pm$ 0.084	9.667 $\pm$ 1.155
V6_L3	0.013 $\pm$ 0.022	0.094 $\pm$ 0.073	0.152 $\pm$ 0.091	29.333 $\pm$ 19.858
V6_L4	0.000 $\pm$ 0.000	0.172 $\pm$ 0.064	0.203 $\pm$ 0.072	51.667 $\pm$ 19.858
V6_L6	0.141 $\pm$ 0.118	0.690 $\pm$ 0.072	0.334 $\pm$ 0.018	187.000 $\pm$ 15.133
V6_L8	0.077 $\pm$ 0.102	0.420 $\pm$ 0.225	0.245 $\pm$ 0.026	126.000 $\pm$ 50.110

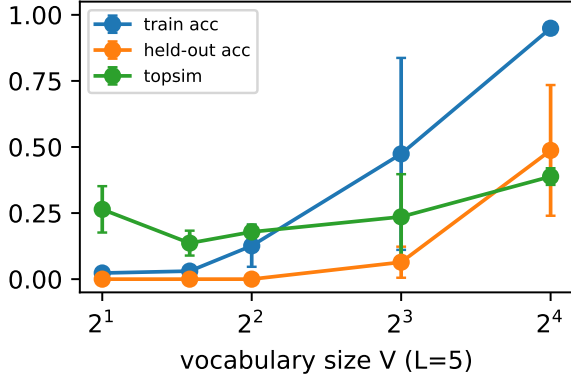


Fig. 3. Train and held-out accuracy and topsim versus vocabulary size ($L = 5$).

codes are fixed senders, not competing methods. Topographic similarity with Hamming distances is only one compositionality measure and is known to be insensitive to some structure; we did not compute others [13]. p -values come from Welch tests with $n = 3$ and are not corrected for multiple comparisons. A full study would add many more seeds, tuned temperature and learning rate (to separate instability from capacity), population or iterated-learning pressures [15], noise [16] and larger object spaces.

VIII. Conclusion

In a small controlled Lewis reconstruction game, learned protocols (Gumbel-softmax and REINFORCE) are far from compositional, with topographic similarity at most 0.388 against 1.000 for an oracle code, and generalise to held-out combinations only in the larger channels we tried (V16_L8, and V16_L5 in the sweep). Capacity below 8 bits clearly prevents learning, but capacity near the minimum is not enough with our untuned settings (Gumbel-softmax peaks early then collapses at V4_L4; REINFORCE memorises without generalising), so the hypothesis that an over-sized channel generalises worse could not be tested fairly and was reversed in our data. The estimators were indistinguishable on held-out accuracy with three seeds. Natural next steps are tuning temperature and learning rate per system, and more seeds.

References

[1] A. Lazaridou, A. Peysakhovich, and M. Baroni, “Multi-Agent Cooperation and the Emergence of (Natural) Language,”

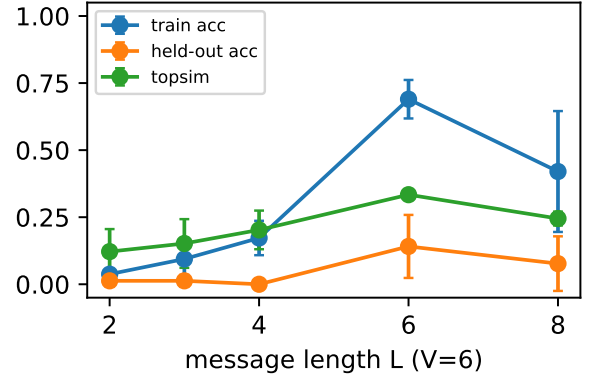


Fig. 4. Train and held-out accuracy and topsim versus message length ($V = 6$).

arXiv preprint arXiv:1612.07182, 2016. [Online]. Available: <https://arxiv.org/abs/1612.07182>

[2] S. Havrylov and I. Titov, “Emergence of Language with Multi-agent Games: Learning to Communicate with Sequences of Symbols,” arXiv preprint arXiv:1705.11192, 2017. [Online]. Available: <https://arxiv.org/abs/1705.11192>

[3] A. Lazaridou and M. Baroni, “Emergent Multi-Agent Communication in the Deep Learning Era,” arXiv preprint arXiv:2006.02419, 2020. [Online]. Available: <https://arxiv.org/abs/2006.02419>

[4] S. Kottur, J. M. F. Moura, S. Lee, and D. Batra, “Natural Language Does Not Emerge ‘Naturally’ in Multi-Agent Dialog,” arXiv preprint arXiv:1706.08502, 2017. [Online]. Available: <https://arxiv.org/abs/1706.08502>

[5] C. Resnick, A. Gupta, J. Foerster, A. M. Dai, and K. Cho, “Capacity, Bandwidth, and Compositionality in Emergent Language Learning,” arXiv preprint arXiv:1910.11424, 2019. [Online]. Available: <https://arxiv.org/abs/1910.11424>

[6] R. Chaabouni, E. Kharitonov, D. Bouchacourt, E. Dupoux, and M. Baroni, “Compositionality and Generalization in Emergent Languages,” arXiv preprint arXiv:2004.09124, 2020. [Online]. Available: <https://arxiv.org/abs/2004.09124>

[7] E. Jang, S. Gu, and B. Poole, “Categorical Reparameterization with Gumbel-Softmax,” arXiv preprint arXiv:1611.01144, 2016. [Online]. Available: <https://arxiv.org/abs/1611.01144>

[8] C. J. Maddison, A. Mnih, and Y. W. Teh, “The Concrete Distribution: A Continuous Relaxation of Discrete Random Variables,” arXiv preprint arXiv:1611.00712, 2016. [Online]. Available: <https://arxiv.org/abs/1611.00712>

[9] R. J. Williams, “Simple statistical gradient-following algorithms for connectionist reinforcement learning,” Machine Learning, 1992. [Online]. Available: <https://doi.org/10.1007/BF00992696>

[10] H. Brighton and S. Kirby, “Understanding Linguistic Evolution by Visualizing the Emergence of Topographic Mappings,” Artificial Life, 2006. [Online]. Available: <https://doi.org/10.1162/artl.2006.12.2.229>

[11] E. Kharitonov, R. Chaabouni, D. Bouchacourt, and M. Baroni, “EGG: a toolkit for research on Emergence of lanGuage in Games,” arXiv preprint arXiv:1907.00852, 2019. [Online]. Available: <https://arxiv.org/abs/1907.00852>

[12] —, “Entropy Minimization In Emergent Languages,” arXiv preprint arXiv:1905.13687, 2019. [Online]. Available: <https://arxiv.org/abs/1905.13687>

[13] T. Korbak, J. Zubek, and J. Raczaszek-Leonardi, “Measuring non-trivial compositionality in emergent communication,” arXiv preprint arXiv:2010.15058, 2020. [Online]. Available: <https://arxiv.org/abs/2010.15058>

[14] O. van der Wal, S. de Boer, E. Bruni, and D. Hupkes, “The Grammar of Emergent Languages,” arXiv preprint

arXiv:2010.02069, 2020. [Online]. Available: <https://arxiv.org/abs/2010.02069>

- [15] Y. Ren, S. Guo, M. Labeau, S. B. Cohen, and S. Kirby, “Compositional Languages Emerge in a Neural Iterated Learning Model,” arXiv preprint arXiv:2002.01365, 2020. [Online]. Available: <https://arxiv.org/abs/2002.01365>
- [16] Łukasz Kuciński, T. Korbak, P. Kołodziej, and P. Miłoś, “Catalytic Role Of Noise And Necessity Of Inductive Biases In The Emergence Of Compositional Communication,” arXiv preprint arXiv:2111.06464, 2021. [Online]. Available: <https://arxiv.org/abs/2111.06464>